

If Else

Content notes, classwork and homework with worked C++ solutions.

Lesson 1: if / if-else Basics

Quick recap: if statement • if-else • Decisions from comparisons

Theory & Nuances

if statement

`if (condition) { ... }` runs the block only when condition evaluates to true. If it's false, the block is simply skipped and the program continues after it — nothing runs in its place.

if-else

Adding an else block gives a second path that runs exactly when the condition is false. Together, if-else guarantees exactly one of the two blocks executes — never both, never neither.

Decisions from comparisons

Conditions are almost always built from the relational/logical operators from the previous topic. Reading an if-else chain is really reading a sequence of true/false questions the program asks about its variables.

Key Points / Common Mistakes

- Without `{ }`, an if only owns the single next statement — adding a second line later without braces is a classic bug.
- A condition that's always true or always false usually signals a logic mistake worth double-checking.

Classwork Problems

1. Odd or even

```
#include <iostream>
using namespace std;

int main() {
    int n = 17;

    if (n % 2 == 0) {
        cout << n << " is even" << endl;
    } else {
        cout << n << " is odd" << endl;
    }
    return 0;
}
```

2. Eligible to vote

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Enter your age: ";
    cin >> age;

    if (age >= 18) {
        cout << "You are eligible to vote." << endl;
    } else {
```

```

        cout << "You are not eligible to vote yet." << endl;
    }
    return 0;
}

```

3. Maximum of two numbers

```

#include <iostream>
using namespace std;

int main() {
    int a = 45, b = 78;

    if (a > b) {
        cout << "Maximum is " << a << endl;
    } else {
        cout << "Maximum is " << b << endl;
    }
    return 0;
}

```

Homework Problems

1. Positive, negative or zero

```

#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter a number: ";
    cin >> n;

    if (n > 0) {
        cout << n << " is positive" << endl;
    } else if (n < 0) {
        cout << n << " is negative" << endl;
    } else {
        cout << "The number is zero" << endl;
    }
    return 0;
}

```

2. Divisible by 5?

```

#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter a number: ";
    cin >> n;

    if (n % 5 == 0) {
        cout << n << " is divisible by 5" << endl;
    } else {
        cout << n << " is not divisible by 5" << endl;
    }
    return 0;
}

```

Lesson 2: Nested if-else / else-if Ladder

Quick recap: Nested if-else • else-if ladder • Multi-way decisions

Theory & Nuances

Nested if-else

An if-else placed inside another if or else block — used whenever a decision only makes sense after a previous decision has already been made. Mentally it's a decision tree: you branch, then branch again inside that branch.

else-if ladder

A chain of else if conditions checked top to bottom. The moment one condition is true, its block runs and every remaining else if / else in the ladder is skipped, even if a later condition would also have been true. Order matters.

Multi-way decisions

When there are more than two possible outcomes (grades, categories, ranges), an else-if ladder is usually far more readable than deeply nested if blocks doing the same job.

Key Points / Common Mistakes

- In an else-if ladder, order conditions from most specific/highest priority to least — the first match wins.
- Nesting more than 2-3 levels deep hurts readability; an else-if ladder (or switch) is often cleaner.

Classwork Problems

1. Maximum of three

```
#include <iostream>
using namespace std;

int main() {
    int a = 23, b = 67, c = 45;
    int maximum;

    if (a > b) {
        if (a > c) maximum = a;
        else maximum = c;
    } else {
        if (b > c) maximum = b;
        else maximum = c;
    }

    cout << "Maximum is " << maximum << endl;
    return 0;
}
```

2. Grade from marks (A/B/C/F)

```
#include <iostream>
using namespace std;

int main() {
    int marks;
    cout << "Enter marks (0-100): ";
    cin >> marks;

    char grade;
    if (marks >= 90) grade = 'A';
    else if (marks >= 75) grade = 'B';
    else if (marks >= 40) grade = 'C';
    else grade = 'F';

    cout << "Grade: " << grade << endl;
    return 0;
}
```

3. Triangle type from its sides

```
#include <iostream>
```

```

using namespace std;

int main() {
    double a = 5, b = 5, c = 5;

    if (a == b && b == c) {
        cout << "Equilateral triangle" << endl;
    } else if (a == b || b == c || a == c) {
        cout << "Isosceles triangle" << endl;
    } else {
        cout << "Scalene triangle" << endl;
    }
    return 0;
}

```

Homework Problems

1. Largest of three distinct numbers

```

#include <iostream>
using namespace std;

int main() {
    int a, b, c;
    cout << "Enter three distinct numbers: ";
    cin >> a >> b >> c;

    int largest;
    if (a > b && a > c) largest = a;
    else if (b > a && b > c) largest = b;
    else largest = c;

    cout << "Largest is " << largest << endl;
    return 0;
}

```

2. Calculator using if-else

```

#include <iostream>
using namespace std;

int main() {
    double a, b;
    char op;

    cout << "Enter expression (e.g. 5 + 3): ";
    cin >> a >> op >> b;

    if (op == '+') {
        cout << "Result: " << (a + b) << endl;
    } else if (op == '-') {
        cout << "Result: " << (a - b) << endl;
    } else if (op == '*') {
        cout << "Result: " << (a * b) << endl;
    } else if (op == '/') {
        if (b != 0) cout << "Result: " << (a / b) << endl;
        else cout << "Error: division by zero" << endl;
    } else {
        cout << "Invalid operator" << endl;
    }
    return 0;
}

```

Lesson 3: Compound Conditions

Quick recap: Compound conditions with &&, ||, ! • Range checks • Multi-condition decisions

Theory & Nuances

Compound conditions

Combining more than one relational check into a single condition lets you express real rules directly — "age is between X and Y", "either A or B, but check C too" — instead of splitting them across several nested ifs.

Range checks

Expressing "between low and high" needs two comparisons joined with `&&` — `x >= 0 && x <= 100`. C++ has no single operator for a math-style range.

Multi-condition decisions

Real eligibility/classification rules often need several `&&` / `||` chained together. De Morgan's-style thinking helps simplify negations: `!(A && B)` is the same as `(!A || !B)`, and `!(A || B)` is the same as `(!A && !B)`.

Key Points / Common Mistakes

- `0 <= x <= 100` compiles in C++ but is **WRONG** — it evaluates `(0<=x)` first into a `1/0`, then compares that to `100`, which is always true. Always write `x >= 0 && x <= 100` instead.
- Parenthesize logical sub-expressions for clarity even when precedence rules don't strictly require it.

Classwork Problems

1. Leap year with branching

```
#include <iostream>
using namespace std;

int main() {
    int year;
    cout << "Enter a year: ";
    cin >> year;

    if (year % 4 == 0) {
        if (year % 100 == 0) {
            if (year % 400 == 0) cout << year << " is a leap year" << endl;
            else cout << year << " is not a leap year" << endl;
        } else {
            cout << year << " is a leap year" << endl;
        }
    } else {
        cout << year << " is not a leap year" << endl;
    }
    return 0;
}
```

2. Classify a character (vowel/consonant/digit)

```
#include <iostream>
using namespace std;

int main() {
    char ch;
    cout << "Enter a character: ";
    cin >> ch;

    if (ch >= '0' && ch <= '9') {
        cout << ch << " is a digit" << endl;
    } else if (ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u' ||
               ch=='A' || ch=='E' || ch=='I' || ch=='O' || ch=='U') {
        cout << ch << " is a vowel" << endl;
    } else if ((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z')) {
        cout << ch << " is a consonant" << endl;
    } else {
        cout << ch << " is not alphanumeric" << endl;
    }
}
```

```
    return 0;
}
```

3. Is a point inside a rectangle?

```
#include <iostream>
using namespace std;

int main() {
    // Rectangle spans x: [0, width], y: [0, height]
    double width = 10, height = 5;
    double x, y;

    cout << "Enter point (x y): ";
    cin >> x >> y;

    if (x >= 0 && x <= width && y >= 0 && y <= height) {
        cout << "Point is inside the rectangle" << endl;
    } else {
        cout << "Point is outside the rectangle" << endl;
    }
    return 0;
}
```

Homework Problems

1. Ticket price by age group

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Enter age: ";
    cin >> age;

    int price;
    if (age < 5) price = 0;
    else if (age >= 5 && age <= 12) price = 50;
    else if (age >= 13 && age <= 59) price = 100;
    else price = 60;

    cout << "Ticket price: Rs " << price << endl;
    return 0;
}
```

2. Valid triangle and its type

```
#include <iostream>
using namespace std;

int main() {
    double a, b, c;
    cout << "Enter three sides: ";
    cin >> a >> b >> c;

    bool isValid = (a + b > c) && (b + c > a) && (a + c > b);

    if (!isValid) {
        cout << "Not a valid triangle" << endl;
    } else if (a == b && b == c) {
        cout << "Valid triangle: Equilateral" << endl;
    } else if (a == b || b == c || a == c) {
        cout << "Valid triangle: Isosceles" << endl;
    } else {
        cout << "Valid triangle: Scalene" << endl;
    }
    return 0;
}
```
